



# LINUX

Complete Textbook with Real-World Analogies

*From Basics to DevOps — Learn with Stories, Not Just Commands*

10 Chapters • Analogies • Diagrams • Command Tables

## □ Table of Contents

---

|                   |                                             |    |
|-------------------|---------------------------------------------|----|
| <b>Chapter 1</b>  | Linux Basics — What, Why & How              | 3  |
| <b>Chapter 2</b>  | File System & Commands — Your File Manager  | 5  |
| <b>Chapter 3</b>  | Users & Permissions — Security Guard System | 8  |
| <b>Chapter 4</b>  | Process Management — Task Manager of Linux  | 11 |
| <b>Chapter 5</b>  | Package Management — App Store of Linux     | 13 |
| <b>Chapter 6</b>  | Networking — How Linux Talks to the World   | 14 |
| <b>Chapter 7</b>  | Disk & Storage — Hard Drive Management      | 16 |
| <b>Chapter 8</b>  | Logs & Monitoring — Linux's Black Box       | 18 |
| <b>Chapter 9</b>  | Shell Scripting — Automate Everything       | 19 |
| <b>Chapter 10</b> | DevOps-Oriented Linux — Production Ready    | 22 |

## Chapter 1

# Linux Basics

Understanding the foundation of the OS

## 1.1 What is Linux?

Linux is an open-source operating system (OS) created by Linus Torvalds in 1991. It is the backbone of servers, cloud computing, Android phones, and supercomputers. Over 90% of the world's servers run Linux.

### ❑ REAL WORLD ANALOGY: Linux vs Windows

- ❑ *Real World: Windows is like a furnished apartment — everything is set up for you, but you can't change the walls. Linux is like an empty flat — you build it exactly the way you want.*
- ❑ *Linux: Linux gives you complete control over every part of the OS. You decide what runs, what's installed, and how resources are used.*

## 1.2 Kernel vs Shell

### ❑ Linux Architecture

|                           |                                 |
|---------------------------|---------------------------------|
| USER APPLICATIONS         | ← Your programs (nano, firefox) |
| SHELL (Bash/Zsh)          | ← Translator / Commander        |
| KERNEL                    | ← Brain of Linux                |
| HARDWARE (CPU, RAM, Disk) | ← Physical Machine              |

### ❑ REAL WORLD ANALOGY: Kernel vs Shell

- ❑ *Real World: A car has an ENGINE (kernel) doing real work — burning fuel, turning wheels. The STEERING WHEEL (shell) is what you use to control it. You never touch the engine directly.*
- ❑ *Linux: Kernel manages hardware. Shell is the interface where you type commands. Shell talks to Kernel, Kernel talks to hardware.*

## 1.3 Linux Distributions

A distribution (distro) is Linux + extra software bundled together. Think of it like different flavors of the same ice cream base.

Debian-Based (apt)

RHEL-Based (yum/dnf)

|                               |                                    |
|-------------------------------|------------------------------------|
| Ubuntu — Beginner friendly    | CentOS — Enterprise stable         |
| Linux Mint — Windows feel     | Red Hat Enterprise Linux (RHEL)    |
| Kali Linux — Security testing | Fedora — Cutting edge features     |
| Command: apt install          | Command: yum install / dnf install |

## 1.4 CLI vs GUI

### ❑ REAL WORLD ANALOGY: CLI vs GUI

- ❑ *Real World: GUI (Graphical Interface) is like an ATM with buttons and touchscreen — easy but limited. CLI (Command Line) is like talking directly to the bank teller — more powerful, faster, and you can do anything.*
- ❑ *Linux: CLI lets you automate, script, and control everything. GUI is slow and limited for servers. Always learn CLI for Linux.*

## 1.5 Important Directories

### ❑ Linux File System Tree

```

/ (Root - The mother of all directories)
├── /etc      → Configuration files (system settings)
├── /var      → Variable files (logs, mail, spool)
├── /home     → User home folders (/home/john, /home/alice)
├── /bin      → Essential commands (ls, cp, mv, cat)
├── /usr      → User programs and applications
├── /tmp      → Temporary files (cleared on reboot)
├── /dev      → Device files (hard drives, USB, etc.)
├── /proc     → Running processes (virtual filesystem)
└── /boot     → Boot files (kernel, grub loader)

```

### ❑ REAL WORLD ANALOGY: Linux Directory Structure

- ❑ *Real World: Think of / (root) as a company building. /home is the employee area, /etc is the manager's config room, /bin is the toolbox, /var is the mailroom that keeps changing.*
- ❑ *Linux: Every file in Linux lives under / — there are no C:/ D:/ drives like Windows.*

# File System & Commands

Navigate, create, copy, move and search files

## 2.1 File Navigation Commands

| Command             | What it does                          | Example                                    |
|---------------------|---------------------------------------|--------------------------------------------|
| <code>pwd</code>    | Print current directory (where am I?) | <code>pwd</code> → <code>/home/john</code> |
| <code>ls</code>     | List files and folders                | <code>ls -la</code> (show hidden files)    |
| <code>ls -la</code> | Detailed list with permissions        | <code>ls -la /etc</code>                   |
| <code>cd</code>     | Change directory                      | <code>cd /var/log</code>                   |
| <code>cd ..</code>  | Go one level up                       | <code>cd ..</code>                         |
| <code>cd ~</code>   | Go to home directory                  | <code>cd ~</code>                          |

## 2.2 File Handling Commands

| Command             | What it does                     | Example                                |
|---------------------|----------------------------------|----------------------------------------|
| <code>cp</code>     | Copy file or folder              | <code>cp file.txt /tmp/</code>         |
| <code>cp -r</code>  | Copy directory recursively       | <code>cp -r /home/john /backup/</code> |
| <code>mv</code>     | Move or rename a file            | <code>mv old.txt new.txt</code>        |
| <code>rm</code>     | Remove a file                    | <code>rm file.txt</code>               |
| <code>rm -rf</code> | Force delete directory (DANGER!) | <code>rm -rf /old_backup</code>        |
| <code>mkdir</code>  | Create new directory             | <code>mkdir myproject</code>           |
| <code>touch</code>  | Create empty file                | <code>touch notes.txt</code>           |

### ❏ REAL WORLD ANALOGY: cp vs mv vs rm

- ❏ Real World: *cp* = Photocopy a document. Original stays, copy goes to new place. *mv* = Actually moving a paper from one drawer to another. *rm* = Shredding the paper permanently.
- ❏ Linux: *rm -rf* is dangerous — no Recycle Bin in Linux! Files deleted with *rm* are gone forever.

## 2.3 File Viewing Commands

| Command          | What it does             | Example                     |
|------------------|--------------------------|-----------------------------|
| <code>cat</code> | Show entire file content | <code>cat /etc/hosts</code> |

|                |                                          |                                               |
|----------------|------------------------------------------|-----------------------------------------------|
| <b>less</b>    | View file page by page (press q to exit) | <code>less /var/log/syslog</code>             |
| <b>head</b>    | Show first 10 lines of file              | <code>head -20 access.log</code>              |
| <b>tail</b>    | Show last 10 lines of file               | <code>tail -f /var/log/syslog</code>          |
| <b>tail -f</b> | Follow file in real-time (live feed)     | <code>tail -f /var/log/nginx/error.log</code> |

#### ❑ **REAL WORLD ANALOGY: tail -f Command**

- ❑ *Real World: Imagine a printer printing pages non-stop. tail -f is like standing next to the printer and reading each page as it comes out — live monitoring.*
- ❑ *Linux: Use tail -f for watching log files in real time. Very useful for debugging production issues.*

## 2.4 Search & Filter Commands

| Command        | What it does                       | Example                                   |
|----------------|------------------------------------|-------------------------------------------|
| <b>find</b>    | Search for files by name/type/size | <code>find /var -name '*.log'</code>      |
| <b>grep</b>    | Search text pattern inside files   | <code>grep 'ERROR' /var/log/syslog</code> |
| <b>grep -r</b> | Recursive search in all files      | <code>grep -r 'password' /etc/</code>     |
| <b>grep -i</b> | Case-insensitive search            | <code>grep -i 'error' app.log</code>      |
| <b>locate</b>  | Fast file search (uses database)   | <code>locate nginx.conf</code>            |
| <b>which</b>   | Find where a command is installed  | <code>which python3</code>                |

#### ❑ **REAL WORLD ANALOGY: find vs grep**

- ❑ *Real World: find is like searching for a BOOK by its title in the library. grep is like opening all books and searching for a WORD inside the content.*
- ❑ *Linux: find locates files by name/properties. grep searches inside file contents for patterns.*

## Chapter 3

# Users & Permissions

*Who can do what — the security system*

### 3.1 User Management

| Command                   | What it does                 | Example                            |
|---------------------------|------------------------------|------------------------------------|
| <code>useradd john</code> | Create new user 'john'       | <code>useradd -m john</code>       |
| <code>passwd john</code>  | Set password for user        | <code>passwd john</code>           |
| <code>usermod</code>      | Modify user settings         | <code>usermod -aG sudo john</code> |
| <code>userdel</code>      | Delete a user                | <code>userdel -r john</code>       |
| <code>groupadd</code>     | Create a new group           | <code>groupadd developers</code>   |
| <code>id</code>           | Show current user and groups | <code>id john</code>               |
| <code>whoami</code>       | Show current logged-in user  | <code>whoami</code>                |
| <code>su - john</code>    | Switch to another user       | <code>su - john</code>             |
| <code>sudo</code>         | Run command as superuser     | <code>sudo apt update</code>       |

#### ❑ REAL WORLD ANALOGY: Users & Groups

- ❑ Real World: A company has EMPLOYEES (users) and DEPARTMENTS (groups). HR group can access HR files. Developers group can access code. The CEO (root) can access everything.
- ❑ Linux: Linux root user = superuser (like CEO). Regular users have limited access. Groups organize users with shared permissions.

### 3.2 Understanding Permissions

#### ❑ Linux Permission Structure

```
$ ls -la myfile.txt
-rwxr-xr-- 1 john developers 4096 May 09 myfile.txt
↑↑↑↑↑↑↑↑
| | | | | Permissions (owner|group|others)
| | | | | File type (- = file, d = directory, l = link)
```

  

```
r = read    (4)    → Can view the file
w = write   (2)    → Can modify the file
x = execute (1)    → Can run as program
```

  

```
rwX r-X r--
7   5   4     ← Octal notation
owner group others
```

### ❏ REAL WORLD ANALOGY: Linux Permissions

❏ *Real World: Imagine 3 types of people at a hotel room: OWNER (you), GROUP (your family), OTHERS (strangers). You can enter, change things, run the TV. Family can enter but not rearrange. Strangers can only look through window.*

❏ *Linux: Owner | Group | Others — each gets Read (r), Write (w), Execute (x) or no permission (-).*

## 3.3 Permission Commands

| Command                       | What it does                  | Example                             |
|-------------------------------|-------------------------------|-------------------------------------|
| <code>chmod 755 file</code>   | Set permissions using numbers | <code>chmod 755 script.sh</code>    |
| <code>chmod +x file</code>    | Add execute permission        | <code>chmod +x deploy.sh</code>     |
| <code>chmod -w file</code>    | Remove write permission       | <code>chmod -w config.txt</code>    |
| <code>chown user file</code>  | Change file owner             | <code>chown john report.txt</code>  |
| <code>chown user:grp</code>   | Change owner and group        | <code>chown john:dev app.py</code>  |
| <code>chgrp group file</code> | Change group only             | <code>chgrp developers code/</code> |

## 3.4 Permission Number Reference

| Number | Permission       | Number | Permission       | Number | Permission  |
|--------|------------------|--------|------------------|--------|-------------|
| 7      | rwX (all)        | 4      | r-- (read)       | 1      | x (execute) |
| 6      | rw- (read+write) | 5      | r-x (read+exec)  | 0      | --- (none)  |
| 3      | wX (write+exec)  | 2      | -w- (write only) |        |             |

### ✔ Quick Memory Trick for chmod Numbers

- `chmod 777` = `rwXrwXrwX` = EVERYONE can do EVERYTHING (dangerous!)
- `chmod 755` = `rwXr-Xr-X` = Owner full, others can read and run (common for scripts)
- `chmod 644` = `rw-r--r--` = Owner read/write, others read-only (common for files)
- `chmod 600` = `rw-----` = Owner only, very private (SSH keys must be 600)



# Process Management

Controlling running programs — Linux's Task Manager

## 4.1 Viewing Processes

| Command                   | What it does                             | Example                          |
|---------------------------|------------------------------------------|----------------------------------|
| <code>ps aux</code>       | Show all running processes               | <code>ps aux   grep nginx</code> |
| <code>top</code>          | Live process monitor (like Task Manager) | <code>top</code>                 |
| <code>htop</code>         | Enhanced interactive process viewer      | <code>htop</code>                |
| <code>pgrep nginx</code>  | Find PID of a process by name            | <code>pgrep -l python</code>     |
| <code>kill PID</code>     | Stop a process by its PID                | <code>kill 1234</code>           |
| <code>kill -9 PID</code>  | Force kill (instant termination)         | <code>kill -9 1234</code>        |
| <code>killall name</code> | Kill all processes with that name        | <code>killall nginx</code>       |

### ❑ REAL WORLD ANALOGY: kill vs kill -9

❑ *Real World: kill = Politely knocking on someone's door and saying 'Please finish and leave.' They may take time. kill -9 = Security guard physically removing them — instant, no questions asked.*

❑ *Linux: Use kill first. Use kill -9 only if the process is frozen and not responding to normal kill.*

## 4.2 Foreground, Background & Daemon Processes

### ❑ Process Types in Linux

#### FOREGROUND PROCESS

└─ Runs in terminal, blocks input until done  
Example: `$ sleep 60` (terminal freezes for 60 sec)

#### BACKGROUND PROCESS

└─ Runs behind terminal, you can keep working  
Example: `$ sleep 60 &` (add `&` to run in background)

#### DAEMON PROCESS

└─ System service, runs continuously in background  
Example: `nginx`, `sshd`, `cron`, `systemd`  
No terminal attached. Starts at boot.

### ❑ REAL WORLD ANALOGY: Foreground vs Background

- ❑ *Real World: Foreground = Cooking at the stove — you must stand there until done. Background = Putting pizza in oven — you can do other things while it cooks. Daemon = Restaurant's HVAC system — runs all the time, you don't think about it.*
- ❑ *Linux: Use & to put process in background. Use fg to bring back. Use jobs to list background tasks.*

| Command                    | What it does                       | Example                             |
|----------------------------|------------------------------------|-------------------------------------|
| <code>command &amp;</code> | Run command in background          | <code>python server.py &amp;</code> |
| <code>jobs</code>          | List all background jobs           | <code>jobs</code>                   |
| <code>fg %1</code>         | Bring job 1 to foreground          | <code>fg %1</code>                  |
| <code>bg %1</code>         | Resume job 1 in background         | <code>bg %2</code>                  |
| <code>Ctrl + Z</code>      | Suspend current foreground process | (keyboard shortcut)                 |
| <code>Ctrl + C</code>      | Kill current foreground process    | (keyboard shortcut)                 |

## Chapter 5

# Package Management

Installing software — Linux's App Store

## 5.1 Debian-Based Systems (Ubuntu)

- ❑ **REAL WORLD ANALOGY: apt Package Manager**
- ❑ *Real World: apt is like Amazon for software. You search, download, and install — all in one command. Linux maintains a list of 'warehouses' (repositories) where software is stored.*
- ❑ *Linux: Always run 'apt update' before 'apt install' to get the latest software list.*

| Command                  | What it does                      | Example                             |
|--------------------------|-----------------------------------|-------------------------------------|
| <code>apt update</code>  | Update list of available packages | <code>sudo apt update</code>        |
| <code>apt upgrade</code> | Upgrade all installed packages    | <code>sudo apt upgrade</code>       |
| <code>apt install</code> | Install a package                 | <code>sudo apt install nginx</code> |
| <code>apt remove</code>  | Remove a package                  | <code>sudo apt remove nginx</code>  |
| <code>apt search</code>  | Search for a package              | <code>apt search python3</code>     |
| <code>dpkg -l</code>     | List all installed packages       | <code>dpkg -l   grep nginx</code>   |

## 5.2 RHEL-Based Systems (CentOS, Red Hat)

| Command                  | What it does                    | Example                             |
|--------------------------|---------------------------------|-------------------------------------|
| <code>yum install</code> | Install package (older systems) | <code>sudo yum install httpd</code> |
| <code>dnf install</code> | Install package (newer systems) | <code>sudo dnf install nginx</code> |
| <code>yum update</code>  | Update all packages             | <code>sudo yum update</code>        |
| <code>dnf remove</code>  | Remove a package                | <code>sudo dnf remove httpd</code>  |
| <code>rpm -qa</code>     | List all installed RPM packages | <code>rpm -qa   grep php</code>     |

| Ubuntu / Debian (apt)               | CentOS / RHEL (dnf/yum)             |
|-------------------------------------|-------------------------------------|
| <code>sudo apt update</code>        | <code>sudo dnf check-update</code>  |
| <code>sudo apt install nginx</code> | <code>sudo dnf install nginx</code> |
| <code>sudo apt remove nginx</code>  | <code>sudo dnf remove nginx</code>  |
| <code>sudo apt upgrade</code>       | <code>sudo dnf upgrade</code>       |
| <code>dpkg -l</code>                | <code>rpm -qa</code>                |

## Chapter 6

# Networking

How Linux talks to the world

## 6.1 Network Information Commands

| Command                    | What it does                           | Example                                   |
|----------------------------|----------------------------------------|-------------------------------------------|
| <code>ip a</code>          | Show all network interfaces and IPs    | <code>ip a</code>                         |
| <code>ifconfig</code>      | Show network config (older command)    | <code>ifconfig eth0</code>                |
| <code>ping</code>          | Test connectivity to a host            | <code>ping google.com</code>              |
| <code>netstat -tuln</code> | Show open ports and listening services | <code>netstat -tuln</code>                |
| <code>ss -tuln</code>      | Modern replacement for netstat         | <code>ss -tuln</code>                     |
| <code>traceroute</code>    | Show path packets take to destination  | <code>traceroute google.com</code>        |
| <code>nslookup</code>      | Query DNS for a domain                 | <code>nslookup google.com</code>          |
| <code>dig</code>           | Advanced DNS lookup                    | <code>dig google.com</code>               |
| <code>curl</code>          | Transfer data from a URL               | <code>curl https://api.example.com</code> |
| <code>wget</code>          | Download files from internet           | <code>wget https://file.iso</code>        |

### ❑ REAL WORLD ANALOGY: IP, Port & DNS — The Addressing System

❑ *Real World: IP Address = Your home address (where to deliver the package). Port = The specific door/department in that building (e.g., front door = 80, mail slot = 25, HR dept = 443). DNS = The phone book that converts 'google.com' to '142.250.80.78'.*

❑ *Linux: IP identifies the server. Port identifies the service (HTTP=80, HTTPS=443, SSH=22, MySQL=3306).*

### ❑ Common Port Numbers

Port 22 → SSH (Secure Shell — remote login)  
Port 80 → HTTP (Web traffic — unencrypted)  
Port 443 → HTTPS (Web traffic — encrypted)  
Port 21 → FTP (File Transfer Protocol)  
Port 25 → SMTP (Email sending)  
Port 3306 → MySQL database  
Port 5432 → PostgreSQL database  
Port 6379 → Redis cache  
Port 8080 → Alternate HTTP (dev servers)

## 6.2 SSH — Secure Shell

### □ REAL WORLD ANALOGY: SSH — Remote Login

- *Real World: SSH is like a secure telephone call to another computer. You speak (type commands) and the remote computer responds — all encrypted so nobody can eavesdrop.*
- *Linux: SSH uses port 22. You can login with password or SSH key (key is more secure — like a physical key vs spoken password).*

| Command                                   | What it does                     | Example                                         |
|-------------------------------------------|----------------------------------|-------------------------------------------------|
| <code>ssh user@host</code>                | SSH into remote server           | <code>ssh john@192.168.1.10</code>              |
| <code>ssh -i key.pem</code>               | SSH with private key file        | <code>ssh -i mykey.pem<br/>ubuntu@server</code> |
| <code>scp file<br/>user@host:/path</code> | Securely copy file to remote     | <code>scp app.zip<br/>john@server:/tmp/</code>  |
| <code>ssh-keygen</code>                   | Generate SSH key pair            | <code>ssh-keygen -t rsa -b 4096</code>          |
| <code>ssh-copy-id</code>                  | Copy public key to remote server | <code>ssh-copy-id john@server</code>            |

## Chapter 7

# Disk & Storage

Managing space, partitions and filesystems

## 7.1 Disk Usage Commands

| Command                           | What it does                               | Example                                |
|-----------------------------------|--------------------------------------------|----------------------------------------|
| <code>df -h</code>                | Show disk space on all filesystems         | <code>df -h</code>                     |
| <code>du -sh /path</code>         | Show size of directory                     | <code>du -sh /var/log</code>           |
| <code>du -sh *</code>             | Show size of each item in directory        | <code>du -sh /home/*</code>            |
| <code>lsblk</code>                | List all block devices (disks, partitions) | <code>lsblk</code>                     |
| <code>fdisk -l</code>             | List disk partition details                | <code>sudo fdisk -l</code>             |
| <code>mount /dev/sdb1 /mnt</code> | Mount a partition to a directory           | <code>mount /dev/sdb1 /mnt/data</code> |
| <code>umount /mnt</code>          | Unmount a partition                        | <code>umount /mnt/data</code>          |
| <code>free -h</code>              | Show RAM and swap memory usage             | <code>free -h</code>                   |

### ❑ REAL WORLD ANALOGY: Partitions and Filesystems

- ❑ *Real World: A hard disk = A big empty plot of land. Partitioning = Dividing land into plots. Formatting = Building foundation on each plot so you can construct buildings. Mounting = Connecting that land to your city map so you can use it.*
- ❑ *Linux: Linux uses ext4 (common), xfs (high-performance), btrfs (modern). Windows uses NTFS. Swap = Virtual RAM on disk when RAM is full.*

### ❑ Disk Troubleshooting — Disk Full Issue

SYMPTOM: 'No space left on device' error

STEP 1: Check which filesystem is full

```
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1       20G   20G   0G   100% /
```

STEP 2: Find what's using space

```
$ du -sh /* 2>/dev/null | sort -rh | head -10
```

STEP 3: Find large files

```
$ find / -size +100M -type f 2>/dev/null
```

STEP 4: Clean old logs

```
$ sudo journalctl --vacuum-size=500M
$ sudo apt autoremove && sudo apt clean
```

## 7.2 Swap Memory

### ❑ REAL WORLD ANALOGY: Swap Memory

❑ *Real World: RAM is like your work desk — limited space but super fast. Swap is like a drawer under the desk — slower but more space. When the desk is full, you put stuff in the drawer temporarily.*

❑ *Linux: Swap = Disk space used as extra RAM. If RAM + Swap are both full, system crashes. Monitor with 'free -h'.*

# Logs & Monitoring

*Linux's Black Box — Everything is recorded*

## 8.1 Important Log Files

| Log File                               | What it contains                                 |
|----------------------------------------|--------------------------------------------------|
| <code>/var/log/syslog</code>           | General system messages (Ubuntu/Debian)          |
| <code>/var/log/messages</code>         | General system messages (RHEL/CentOS)            |
| <code>/var/log/auth.log</code>         | Authentication logs — login attempts, sudo usage |
| <code>/var/log/kern.log</code>         | Kernel messages — hardware issues                |
| <code>/var/log/nginx/access.log</code> | All HTTP requests to nginx web server            |
| <code>/var/log/nginx/error.log</code>  | Nginx errors — 404s, crashes, config issues      |
| <code>/var/log/mysql/error.log</code>  | MySQL database errors                            |
| <code>/var/log/cron</code>             | Cron job execution logs                          |

### ❑ REAL WORLD ANALOGY: System Logs

- ❑ *Real World: Logs are like an airplane's BLACK BOX. You don't watch it during normal flight. But when something goes wrong, you open it to find exactly what happened and when.*
- ❑ *Linux: Never ignore logs in production. Logs tell you what failed, when it failed, and sometimes why — before users complain.*

## 8.2 Log Monitoring Commands

| Command                              | What it does             | Example                                       |
|--------------------------------------|--------------------------|-----------------------------------------------|
| <code>tail -f /var/log/syslog</code> | Watch log in real-time   | <code>tail -f /var/log/nginx/error.log</code> |
| <code>journalctl</code>              | View systemd logs        | <code>journalctl -u nginx</code>              |
| <code>journalctl -f</code>           | Follow live systemd logs | <code>journalctl -f</code>                    |
| <code>journalctl -n 50</code>        | Show last 50 log entries | <code>journalctl -n 100 -u sshd</code>        |
| <code>grep 'ERROR' syslog</code>     | Filter specific errors   | <code>grep -i 'fail' /var/log/auth.log</code> |



## Chapter 9

# Shell Scripting

*Automate repetitive tasks with bash scripts*

## 9.1 Variables

Variables store data that can be reused. Think of it like labeling a box: instead of writing the contents every time, you refer to it by name.

### □ Variables in Shell Script

```
#!/bin/bash
# This is a comment

NAME='John'          # String variable
AGE=25               # Number variable
TODAY=$(date)        # Store command output in variable

echo 'Hello, $NAME'   # Output: Hello, $NAME (single quotes = literal)
echo "Hello, $NAME"   # Output: Hello, John (double quotes = expand)
echo "Today is $TODAY"
```

## 9.2 If Conditions

### □ REAL WORLD ANALOGY: If Conditions

- Real World: Traffic light logic: IF light is green → drive. ELSE IF light is yellow → slow down. ELSE (red) → stop. Conditions work exactly the same in scripts.
- Linux: Shell scripting uses if/elif/else/fi to make decisions based on conditions.

### □ If-Else Example

```
#!/bin/bash
DISK_USAGE=$(df / | awk 'NR==2 {print $5}' | tr -d '%')

if [ $DISK_USAGE -gt 90 ]; then
    echo 'CRITICAL: Disk is above 90%! Clean up now!'
elif [ $DISK_USAGE -gt 70 ]; then
    echo 'WARNING: Disk usage is $DISK_USAGE%'
else
    echo 'OK: Disk usage is fine at $DISK_USAGE%'
fi
```

## 9.3 Loops

#### ❑ For Loop Example

```
#!/bin/bash
# Loop through servers and ping each one
SERVERS='web1 web2 web3 db1'

for SERVER in $SERVERS; do
    echo "Checking $SERVER..."
    ping -c 1 $SERVER > /dev/null && echo '$SERVER is UP' || echo '$SERVER
is DOWN'
done
```

#### ❑ While Loop Example

```
#!/bin/bash
# Count from 1 to 5
COUNT=1

while [ $COUNT -le 5 ]; do
    echo "Count: $COUNT"
    COUNT=$((COUNT + 1))
done
echo 'Done counting!'
```

## 9.4 Functions

#### ❑ Functions in Shell Script

```
#!/bin/bash
# Define a reusable function
check_service() {
    SERVICE=$1                # $1 = first argument
    if systemctl is-active --quiet $SERVICE; then
        echo '$SERVICE is RUNNING'
    else
        echo '$SERVICE is STOPPED - restarting...'
        systemctl restart $SERVICE
    fi
}

# Call the function with different services
check_service nginx
check_service mysql
check_service redis
```

## 9.5 Cron Jobs — Scheduled Tasks

#### ❑ REAL WORLD ANALOGY: Cron Jobs



Chapter 10

# DevOps-Oriented Linux

*Production skills every DevOps engineer needs*

## 10.1 Systemctl — Service Management

### ❑ REAL WORLD ANALOGY: systemctl

- ❑ *Real World: systemctl is like the control panel of a factory. Each machine (service) can be turned ON (started), turned OFF (stopped), set to auto-start (enabled), or checked for status.*
- ❑ *Linux: systemctl manages all Linux services: nginx, mysql, docker, sshd, cron, etc.*

| Command                              | What it does                | Example                                          |
|--------------------------------------|-----------------------------|--------------------------------------------------|
| <code>systemctl start nginx</code>   | Start a service             | <code>sudo systemctl start nginx</code>          |
| <code>systemctl stop nginx</code>    | Stop a service              | <code>sudo systemctl stop mysql</code>           |
| <code>systemctl restart nginx</code> | Restart a service           | <code>sudo systemctl restart nginx</code>        |
| <code>systemctl status nginx</code>  | Check if service is running | <code>systemctl status sshd</code>               |
| <code>systemctl enable nginx</code>  | Auto-start on boot          | <code>sudo systemctl enable nginx</code>         |
| <code>systemctl disable nginx</code> | Don't auto-start on boot    | <code>sudo systemctl disable nginx</code>        |
| <code>systemctl list-units</code>    | List all active services    | <code>systemctl list-units --type=service</code> |

## 10.2 Environment Variables

| Command                       | What it does                   | Example                               |
|-------------------------------|--------------------------------|---------------------------------------|
| <code>export VAR=value</code> | Set an environment variable    | <code>export DB_HOST=localhost</code> |
| <code>echo \$VAR</code>       | Print a variable's value       | <code>echo \$PATH</code>              |
| <code>env</code>              | List all environment variables | <code>env   grep DB</code>            |
| <code>printenv VAR</code>     | Print specific env variable    | <code>printenv HOME</code>            |
| <code>unset VAR</code>        | Remove an environment variable | <code>unset OLD_CONFIG</code>         |
| <code>source .env</code>      | Load .env file into shell      | <code>source /etc/environment</code>  |

## 10.3 Firewall Management

| firewalld (RHEL/CentOS)                       | ufw (Ubuntu/Debian) |
|-----------------------------------------------|---------------------|
| firewall-cmd --list-all                       | ufw status          |
| firewall-cmd --add-port=80/tcp --permanent    | ufw allow 80/tcp    |
| firewall-cmd --add-service=http --permanent   | ufw allow http      |
| firewall-cmd --reload                         | ufw reload          |
| firewall-cmd --remove-port=80/tcp --permanent | ufw deny 80/tcp     |

## 10.4 CPU High Usage — Troubleshooting

### □ CPU High Issue — Step by Step

SYMPTOM: Server is very slow, CPU at 100%

STEP 1: Identify which process is using CPU

```
$ top    OR    $ htop
(Press 'P' in top to sort by CPU %)
```

STEP 2: Get more info about that process

```
$ ps aux | grep <process name>
$ lsof -p <PID>    (see what files it has open)
```

STEP 3: Check logs for that service

```
$ journalctl -u <service name> -n 100
```

STEP 4: Options — restart, kill, or fix config

```
$ sudo systemctl restart <service_name>
$ kill -9 <PID>    (last resort)
```

STEP 5: Monitor to confirm CPU returns to normal

```
$ watch -n 2 'ps aux --sort=-%cpu | head -10'
```

## 10.5 SSH Configuration

### □ SSH Config File — /etc/ssh/sshd\_config

```
# Disable root SSH login (security best practice)
PermitRootLogin no
```

```
# Change default SSH port (security through obscurity)
Port 2222
```

```
# Disable password auth — use SSH keys only
PasswordAuthentication no
```

```
# Allow specific users only
AllowUsers john alice devops
```

```
# Set max login attempts
MaxAuthTries 3

# After editing, restart sshd
# sudo systemctl restart sshd
```

### ✓ DevOps Linux Best Practices — Remember These!

- Never run applications as root — create a dedicated user (e.g., 'appuser')
- SSH keys only — disable password login in production servers
- Always monitor with: top, df -h, free -h after deployment
- Use tail -f on log files when debugging live issues
- Set up cron jobs for automated backups and log rotation
- chmod 600 for private keys, chmod 755 for scripts, chmod 644 for config files
- systemctl enable <service> so services survive server reboots
- grep and awk are your best friends for parsing log files quickly

# 📁 LINUX QUICK REFERENCE CHEAT SHEET

*The most important commands — memorize these!*

| 📁 Files & Navigation           | 👤 Users & Permissions   |                                  |                 |
|--------------------------------|-------------------------|----------------------------------|-----------------|
| <code>ls -la</code>            | List files with details | <code>chmod 755 file</code>      | Set permissions |
| <code>cd /path</code>          | Change directory        | <code>chown user:grp file</code> | Change owner    |
| <code>pwd</code>               | Current directory       | <code>useradd john</code>        | Create user     |
| <code>cp -r src dst</code>     | Copy directory          | <code>passwd john</code>         | Set password    |
| <code>mv old new</code>        | Move or rename          | <code>sudo command</code>        | Run as root     |
| <code>rm -rf dir</code>        | Delete directory        | <code>id username</code>         | Show user info  |
| <code>find / -name file</code> | Search for file         | <code>groups john</code>         | Show groups     |
| <code>grep 'text' file</code>  | Search in file          | <code>usermod -aG grp usr</code> | Add to group    |

| 📁 Process & Services          | 🌐 Networking & Disk  |                              |                   |
|-------------------------------|----------------------|------------------------------|-------------------|
| <code>ps aux</code>           | Show all processes   | <code>ip a</code>            | Show IP addresses |
| <code>top / htop</code>       | Live process monitor | <code>ping google.com</code> | Test connectivity |
| <code>kill -9 PID</code>      | Force kill process   | <code>ssh user@host</code>   | Remote login      |
| <code>systemctl start</code>  | Start a service      | <code>curl URL</code>        | Fetch from URL    |
| <code>systemctl status</code> | Check service status | <code>df -h</code>           | Disk space usage  |
| <code>systemctl enable</code> | Auto-start on boot   | <code>du -sh /path</code>    | Directory size    |
| <code>journalctl -f</code>    | Follow live logs     | <code>free -h</code>         | RAM + Swap usage  |
| <code>crontab -e</code>       | Edit cron jobs       | <code>lsblk</code>           | List disks        |

## 📁 Linux Mastery — Practice is the key!

*Read the concept → Understand the analogy → Type the command → Remember forever*